New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

Digital Asset Standard (DAS)

Solana DAS API: Unified NFT and Token Data Access

Copy page

The most comprehensive Solana API for NFTs, compressed NFTs, and tokens. Unified interface for all digital assets with high performance and complete metadata.

Quick Reference: The DAS API provides a unified interface for all Solana assets. Use `getAsset` for single assets, `getAssetsByOwner` for wallet holdings, `searchAssets` for filtered queries, and specialized methods for compressed NFTs.

What is DAS?

The Digital Asset Standard (DAS) API is an open-source specification that provides a unified interface for interacting with all types of digital assets on Solana:

- Regular NFTs (Non-Fungible Tokens)
- Compressed NFTs (State Compression)
- Fungible Tokens (including SPL Tokens and Token-2022)
- Inscriptions and SPL-20 tokens (experimental)

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

https://mainnet.helius-rpc.com/?api-key=YOUR_API_KEY

https://devnet.helius-rpc.com/?api-key=YOUR_API_KEY

Key Features | Use Cases

- Unified access to all Solana asset types
- Comprehensive metadata retrieval
- Off-chain data indexing (Arweave, IPFS)
- Merkle proof support for compressed assets
- Token price data for verified tokens
- Advanced filtering and search capabilities

API Methods

Single Asset
Get detailed data for specific assets

Asset Collections
Get assets by owner, creator, or collection

Advanced Queries
Search and filter assets with complex criteria

Fetching Individual Assets

These methods let you retrieve detailed information about specific assets by their IDs.

getAsset | getAssetBatch | getAssetProof | getAssetProofBatch

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

HELIUS

```
const url = `https://mainnet.helius-rpc.com/?api-key=YOUR_API_KEY`;

const getAsset = async (id) => {
  const response = await fetch(url, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      jsonrpc: "2.0",
      id: "my-request-id",
      method: "getAsset",
      params: {
        id: id,
      },
    }),
  });

  const data = await response.json();
  return data.result;
};

// Example usage
getAsset("FNt6A9Mfnqbwc1tY7uwAguKQ1JcpBrxmhczDgbdJy5AC");
```

API Reference
View detailed documentation for getAsset

Fetching Asset Collections

These methods allow you to retrieve assets based on ownership, creation, authority, or collection membership.

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

HELIUS

By Owner By Collection By Creator By Authority

```
// Get all assets owned by a wallet address
const url = `https://mainnet.helius-rpc.com/?api-key=YOUR_API_KEY`;

const getAssetsByOwner = async (ownerAddress) => {
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      jsonrpc: '2.0',
      id: 'my-request-id',
      method: 'getAssetsByOwner',
      params: {
        ownerAddress: ownerAddress,
        page: 1,
        limit: 1000,
        displayOptions: {
          showFungible: true, // Include SPL tokens
          showNativeBalance: true, // Include SOL balance
          showInscription: true, // Include inscription data
        },
      },
    }),
  });

  const { result } = await response.json();
  return result;
};

// Example usage
getAssetsByOwner("86xCnPeV69n6t3DnyGvkKobf9FdN2H9oiVDdaMpo2MMY");
```

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

View detailed documentation for getAssetsByOwner

Advanced Query Methods

These methods allow for more specialized asset queries, including transaction history, NFT editions, and complex search filters.

searchAssets | getSignaturesForAsset | getNftEditions | getTokenAccounts

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

HELIUS

```
const url = `https://mainnet.helius-rpc.com/?api-key=YOUR_API_KEY`;

const searchAssets = async (searchParams) => {
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      jsonrpc: '2.0',
      id: 'my-request-id',
      method: 'searchAssets',
      params: searchParams,
    }),
  });

  const { result } = await response.json();
  return result;
};

// Example: Search for NFTs in a collection
searchAssets({
  ownerAddress: "86xCnPeV69n6t3DnyGvkKobf9FdN2H9oiVDdaMpo2MMY",
  grouping: ["collection", "J1S9H3QjnRtBbbuD4HjPV6RpRhwuk4zKbxsnCHu
  limit: 100
});
```

API Reference
View detailed documentation for searchAssets

Working with Special Asset Types

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

Access SPL tokens and get mint data

Work with state-compressed assets

Inscriptions

Access inscription and SPL-20 data

Off-chain Data

Retrieve metadata from Arweave/IPFS

Fungible Tokens

Helius has extended the DAS API to support **all tokens**, including plain SPL tokens (without metadata) and Token-2022 (plus their extensions). Fungible token support is available through the `getAssetsByOwner` and `searchAssets` methods.

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

HELIUS

```
const url = `https://mainnet.helius-rpc.com/?api-key=YOUR_API_KEY`;

const getTokensWithPrices = async (ownerAddress) => {
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      jsonrpc: '2.0',
      id: 'my-request-id',
      method: 'getAssetsByOwner',
      params: {
        ownerAddress: ownerAddress,
        displayOptions: {
          showFungible: true,
        },
      },
    }),
  });

  const { result } = await response.json();

  // Filter tokens that have price data
  const tokensWithPrices = result.items.filter(
    asset => asset.token_info?.price_info
  );

  return tokensWithPrices;
};

// Example usage
getTokensWithPrices("86xCnPeV69n6t3DnyGvkKobf9FdN2H9oiVDdaMpo2MMY")
```

Compressed NFTs

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

retrieving Merkle proofs and transaction history.

Getting Compressed NFT Data

›

```
const url = `https://mainnet.helius-rpc.com/?api-key=YOUR_API_KEY`;

const getCompressedNft = async (id) => {
  const response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      jsonrpc: '2.0',
      id: 'my-request-id',
      method: 'getAsset',
      params: { id },
    }),
  });

  const { result } = await response.json();
  return result;
};
```

Retrieving Merkle Proofs

Merkle proofs are required for verifying and performing on-chain operations with compressed NFTs like transfers and burns.

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More


```
  const response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      jsonrpc: '2.0',
      id: 'my-request-id',
      method: 'getAssetProof',
      params: { id },
    }),
  });

  const { result } = await response.json();
  return result;
};
```

Checking Transaction History

```
const getTransactionHistory = async (id) => {
  const response = await fetch(url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      jsonrpc: '2.0',
      id: 'my-request-id',
      method: 'getSignaturesForAsset',
      params: {
        id,
        page: 1,
        limit: 100
      },
    }),
  });

  const { result } = await response.json();
  return result;
};
```

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

You can optionally display inscription and SPL-20 token data with the `showInscription` flag. This is an experimental feature.

›

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

HELIUS

```
const url = `https://mainnet.helius-rpc.com/?api-key=YOUR_API_KEY`;

const getInscriptionsForWallet = async (ownerAddress) => {
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      jsonrpc: '2.0',
      id: 'my-request-id',
      method: 'getAssetsByOwner',
      params: {
        ownerAddress: ownerAddress,
        displayOptions: {
          showInscription: true,
        },
      },
    }),
  });

  const { result } = await response.json();

  // Filter results to only include assets with inscription data
  const inscriptions = result.items.filter(
    asset => asset.inscription
  );

  return inscriptions;
};

// Example usage
getInscriptionsForWallet("6GmTFg5SCs4zGfDEidUAJjS5pSrXEPwW8Rpfs3RHr
```

Off-chain Data

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

Pagination

- Start at page 1 (not 0)
- Use reasonable limit values (100-1000)
- Implement robust pagination for large datasets

Error Handling

- Always check for errors in responses
- Implement retries with exponential backoff
- Handle edge cases like invalid parameters

Performance

- Use batch methods when possible
- Cache responses when appropriate
- Avoid chaining multiple requests when one will do

Security

- Never expose your API key in client-side code
- Use server-side functions to proxy API requests
- Set up appropriate CORS policies

Resources

API References

GitHub Repository

Examples & SDKs

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More

documentation for all DAS methods

source specification

client libraries and code samples

Was this page helpful?

Yes No Suggest edits Raise issue

New: Try Gatekeeper (Beta) for significantly lower latency. Learn More